CS4641/7641 · TEXT MINING · PROJECT REPORT

Yelp Review Analysis: From Baselines to Business Intelligence Agent

A four-stage NLP pipeline progressing from feature engineering to agentic RAG systems

GitHub: github.com/YuhanRongGT/Yelp Project

Project Overview

This project applies a progressive four-stage NLP pipeline to the Yelp Academic Dataset (50,000 reviews). Each stage builds on the previous, exploring feature performance and capability evolve as methods advance from traditional feature engineering to a fully agentic retrieval augmented system.

Stage 1	Stage 2	Stage 3	Stage 4
Baseline Comparison	**Feature Ablation**	**LLM Evaluation**	**LLM Agent**
TF-IDF + Lexicon Engineering, BERT fine-tuning	5 experiments across 3 hardware tiers, ablation study	Zero-shot, few-shot, LoRA fine-tuning with Qwen 8B	Tool-calling ReAct agent with three-step retrieval

Key progression: feature engineering → Pretrained representations → End-to-end transformer fine-tuning → Direct LLM prompting → Agentic retrieval augmented systems

Stage 1 — Baseline Comparison

Three modeling paradigms are compared on the 5-class star rating prediction task (test set).

Method	Accuracy	Macro-F1
TF-IDF + Logistic Regression	56.92%	56.10%
Sentence-BERT + Logistic Regression	53.25%	52.09%
BERT (fine-tuned)	**62.58%**	**62.10%**

Why does fine-tuned BERT win? End-to-end training jointly optimizes the text representation and the classification boundary for this task, while TF-IDF relies on sparse lexical features and Sentence-BERT uses a frozen embedding space. Ordinal regression and cost-sensitive learning objective did not improve performance, suggesting the standard cross-entropy objective and class-weighting combination cannot be corrected.

Why does Sentence-BERT underperform TF-IDF? Sentence-BERT embeddings are optimized for semantic similarity between sentence pairs, not for single-review classification. Star ratings are primarily driven by specific sentiment vocabulary ("terrible", "amazing", "overpriced") — patterns that TF-IDF's word-frequency statistics capture directly. Sentence-BERT compresses them into a dense vector designed for different purposes.

Stage 2 — BERT Fine-Tuning Experiments

A systematic hyperparameter search across **216 configurations** covers all combinations of the following dimensions:

Dimension	Values
Backbone model	distilroberta-base, uncased , bert-base-uncased , roberta-base
Text setting	Binary (2 class), 5 class, 5 class
Loss function	Classification cross-entropy, Regression (MSE)
Learning rate	1e-5, 2e-5
Batch size	4, 8, 16
Seed	1, 2, 3
Total combinations	**3 × 3 × 2 × 2 × 2 × 3 = 216**

Each sum seven validation and test metrics, classification reports, confusion matrices, and predictions files. Due to compute constraints, hyperparameters, model checkpoints are not saved during training. Only with-validation outputs are retained. The best 5-class configuration is re-trained and saved in Stage 4 (Step 0).

BERT (fine-tuned)

Metric	Test	Best Hyperparameters	Rank
DistilBERT base uncased	56.04%	16,32,5e-	—
BERT base uncased	62.58%	16,16,5e-	1st
RoBERTa base	**57.82%**	**57.81%**	**0.39bb**

5 class — test set

Model	Accuracy	Macro-Recall	Macro-F1
DistilBERT base uncased	65.45%	65.36%	57.25%
BERT base uncased	66.75%	66.30%	57.49%
RoBERTa base	**68.56%**	**68.49%**	**59.10%**

Off-by-1 Accuracy explained: For star rating prediction, a prediction is counted as "off-by-1 correct" if it falls within ±1 of the ground truth. Predicting 4 when the true label is 3 or 5 ✓, but predicting 1 when the true label is 3 ✗. This metric captures near-miss predictions that are practically acceptable — a user who rates a restaurant 4 stars and a model that predicts 3 stars is more useful than one predicting 1 star. Exact accuracy treats all errors equally, off-by-1 accuracy distinguishes between minor and catastrophic errors. Useful when the 5-class task is ordinal and adjacent classes are confusing to a human.

Why is 3 class so much better than binary? Binary classification separates clearly negative (1–2★) from clearly positive (4–5★) reviews, where sentiment signals are strong. 3 class requires distinguishing adjacent ratings (e.g., 2★ vs 3★) which often share similar vocabulary. The jump from Off-by-1 Accuracy (≈97%) confirms the model rarely confuses distant classes — the bottleneck is the internal ambiguity between neighboring star levels.

Best configuration (3 class): RoBERTa base · LR=2e-5 · batch=4 · epochs=4. This checkpoint is reused as the review classifier tool in Stage 4.

Stage 3 — LLM Evaluation and Fine-Tuning

Two instruction-tuned LLMs are evaluated on the same rating task through a three-tier progression.

Qwen3.5-9B Instruct	Deepseek-R1-Distill-Qwen-14B
→ Tier 1: Zero-shot direct	→ Tier 1: Zero-shot direct
→ Tier 2: LoRA fine-tuning (QLoRA)	→ Tier 2: Few-shot chain-of-thought prompting
	→ Tier 3: LoRA fine-tuning + thinking chains

Fine-Tuning Details

Qwen3.5-9B Instruct — two hardware configurations

Parameter	20GB Unsloth (local)	5090 (RTX PRO 6000)
Quantization	4bit QLoRA (quant type: bitsandb compute)	None + full bf16
LoRA rank r	16	32
LoRA alpha	32	64
LoRA dropout	0.05	0.05
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj	(7 modules)
Learning rate	2e-4	2e-4
Epochs	2	2
Batch size (effective)	4 × 4 = 16	8 × 8 = 32
Gradient checkpointing	Unsloth offloading	True
Max seq length	512 (tokens truncated)	1024
Reported results	—	2 higher throughput

Both configurations apply LoRA adapters to all 7 projection layers (attention + FFN) gate/up/down). Sequence packing is enabled for training to eliminate padding waste. The model is trained with cross-entropy loss. Inference uses a separate 1024 token context window to avoid truncating long Yelp reviews.

Deepseek-R1-Distill-Qwen-14B — two stage fine-tuning ablation

Fine-tuning a reasoning model requires preserving its chain-of-thought structure. A dedicated two-stage pipeline is used:

Stage	Purpose	Output
Stage 1 — Generate thinking chains	Run DeepSeek on the training set, extract <think>...</think> traces from correctly parsed predictions	Structured CoT with reasoning chains
Stage 2 — LoRA fine-tuning	Fine-tune on thinking chains data with conversation format	LoRA adapter weights

Hyperparameters to preserve reasoning ability

LoRA hyperparameters for DeepSeek fine-tuning:

Parameter	Value	Rationale
LoRA rank r	16	Conservative — smaller rank reduces risk of overwriting reasoning structure
LoRA alpha	32	Standard 2× ratio
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj	Same 7 modules coverage as Qwen
Learning rate	1e-5	4× lower than Qwen — preserves reasoning ability in distilled model
Epochs	2	—
Batch size (effective)	4 × 4 = 16	—
Max seq len	1024	Thinking chains are much longer than plain reviews, 256 would truncate them

Training format for the assistant turn:

```
<think>
{thinking_content reasoning_chain}
</think>
{"useful": 2, "funny": 0, "cool": 0}
```

Only samples where the base model already predicted the correct label are included — incorrect thinking chains are discarded to avoid reinforcing wrong reasoning.

DeepSeek Prompt Engineering — Tier 2 Improvement

DeepSeek-R1 Distill tends to be intermediate prompting sits between Tier 1 (Zero-shot Qwen) and Tier 3 (fine-tuned performance was severely limited by an insufficient token budget and the absence of anchor examples. The following changes raised 5-class accuracy from 15.60% to 67.56%:

Setting	Baseline Prompt	Improved Prompt
max_tokens	128	512
Few-shot examples	None (zero-shot)	3 examples (one per low-mid high)
Reasoning instruction	Unrestricted	"Reason briefly (2-3 sentences)"
Total prediction rate	65.4%	~99%
5-class accuracy	15.60%	67.56%

Baseline prompt (max_tokens=128, zero-shot):

```
You are a Yelp review rating classifier.
Analyze the review carefully.
The label must be an integer from 1 to 5.
You may reason first, but the final line must be exactly:
Rating: <1|2|3|4|5>
Where 1=★, 2=★★, 3=★★★, 4=★★★★, 5=★★★★★.

Review: {review_text}
```

Improved prompt (max_tokens=512, 3 shot examples, brief reasoning):

```
You are a Yelp review rating classifier.
Analyze carefully. If unsure, default to adjacent stars.
Your LAST line must be exactly: Final rating: 1| 2| 3| 4|5

Here are 3 examples:
Review: "Absolutely amazing! Best experience ever!" → Final rating: 5
Review: "Food was okay, service was okay." → Final rating: 3
Review: "Terrible, waiting 45 min!" → Final rating: 1
Review: "Decent enough for the price." → Final rating: 3
Review: "Not bad, but nothing special." → Final rating: 3
Review: "Horrible! Never coming back!" → Final rating: 1

Review: {review}
```

Why did each change improve accuracy?

1. **max_tokens 128 → 512:** DeepSeek-R1's chain-of-thought alone typically uses 100–300 tokens. With only 128 tokens, ~35% of responses were truncated before the "Final rating:" line ever appeared, causing parsing failures. The budget to 512 directly recovered these responses.
2. **3 shot examples (one per low/mid/high):** Zero-shot prompts leave the model uncertain about how each star level maps to review tone. Providing one calibrated example per level anchors the scale and forces consistent output format, reducing both format errors and label bias.
3. **Brief reasoning (2–3 sentences):** Without a length constraint, the reasoning chain can still grow long enough to exhaust 512 tokens before the final answer. Capping reasoning to 2–3 sentences ensures the label always appears within budget.

Binary (2 class) — test set

Model	Method	Accuracy	Macro-F1
Qwen3.5-9B Instruct	LoRA fine-tuned (20GB)	98.56%	98.56%
Qwen3.5-9B Instruct	LoRA fine-tuned (5090)	98.55%	98.55%
DeepSeek-R1-14B	Zero-shot	80.04%	79.42%
DeepSeek-R1-14B	Few-shot (improved)	97.22%	97.22%
DeepSeek-R1-14B	LoRA fine-tuna (5090)	98.42%	98.42%

5 class — test set

Model	Method	Accuracy	Macro-F1
Qwen3.5-9B Instruct	LoRA fine-tuna (5090)	69.39%	68.25%
Qwen3.5-9B Instruct	Zero-shot	67.36%	66.91%
DeepSeek-R1-14B	LoRA fine-tuna (5090)	67.70%	66.45%
DeepSeek-R1-14B	Few-shot (improved)	67.56%	66.90%
DeepSeek-R1-14B	Zero-shot	55.60%	54.49%

Why is Qwen's zero-shot already near fine-tuned? Qwen3.5-7B instruct was pre-trained on a large corpus including instruction-following data. It already understands the relationship between review sentiment and star ratings without task-specific training. The marginal gain from LoRA fine-tuning (+0.19% on binary) reflects task adaptation — the model's capability ceiling for this task is reached at zero-shot.

Why did DeepSeek fail at zero-shot? DeepSeek-R1 is a reasoning model trained to think step-by-step before answering. On a classification task, it applies chain-of-thought reasoning to star rating prediction, and often overthinks and reaches a final label. This is a positive engineering failure, not a capability failure — after increasing the token budget and adding few-shot examples, 5-class accuracy jumped from 15.6% to 67.6%.

Why does LoRA fine-tuning work effectively? LoRA freezes the base model weights and trains only small low-rank update matrices, preserving pre-trained language knowledge while adapting the output format. For strong base models (Qwen), this preserves instruction-following capability while teaching the exact output schema. For reasoning models (DeepSeek), it preserves reasoning ability while teaching the classification task.

Why use thinking chain data for DeepSeek fine-tuning? Directly fine-tuning DeepSeek on (input → label) pairs would suppress its chain-of-thought structure, degrading reasoning ability. By generating thinking chains from correct predictions first and training on (input → chain + <think>...</think> → label), the model learns to reproduce its reasoning while learning the correct output format.

Stage 4 — Business Intelligence Agent

Stage 4 shifts from classification to open-ended question answering over Yelp reviews. A tool-calling agent system is built and evaluated against a no retrieval LLM baseline.

System Architecture

Agent Tools

Tool	Description
get_business_info	Returns review count, average stars, and distribution for a given business.
search_review_chunks_by_business	Semantic search restricted to a single business (pre-filtered HNSW query)
search_review_chunks_global	Global semantic search across all 50,000 Yelp review chunks
summarize_evidence	Calls Qwen on Ollama to synthesize retrieved chunks into structured findings.
classify_review	Runs the Stage 2 RoBERTa 5-class classifier on a review text.

The agent has access to all tools unconditionally, giving a 7B model one focused tool per scope significantly improves tool selection reliability compared to a single multi-purpose retrieval tool.

Technical Stack

Component	Implementation	Why
Tool-calling loop	Qwen3.5-7B via Ollama	Native function calling support
Retrieval	ChromaDB + BGE-small-en-v1.5	Persistent HNSW index, cosine similarity
Reranker	BGE-reranker-base	Cross-encoder reranking improves precision over bi-encoder retrieval
Structured output	Pydantic v2	Validates tool call arguments and agent responses
Tracing	Langfuse + langfuse-langchain	Logs every LLM call, tool call, latency, and token usage

Vector Knowledge Base

- 10,000 Yelp reviews → **88,632 review chunks** (sentence-split, ≤180 words each)
- Encoded with `all-MiniLM-L6-v2` (384-dim, L2-normalized → cosine similarity via inner product)
- Stored in ChromaDB with HNSW cosine index
- Business filtered retrieval: subset matrix multiply on `business_to_indices` — exact results, no approximation for small businesses

Evaluation Suite

20 questions × 4 types × 3 systems = 60 evaluated responses. Human scored on 3 dimensions (0–2 scale, max 6).

Question Type	Example
Factual	"What do customers complain about most at this business?"
Aspect Analysis	"What aspects do customers praise and criticize about food and service?"
Business Profiling	"Give me an overall profile of this business based on customer reviews."
Cross-Business Pattern	"Do Italy reviews show any patterns between neat time and star ratings?"

Evaluation Results

System	Correctness	Evidence	Groundedness	Tool Use	Efficiency	Total /6
Direct LLM	0.25	0.00	0.00	1.00	1.10	1.35
RAG Baseline	**0.95**	**1.40**	**1.15**	**1.00**	**1.10**	**5.60**
Agent	1.05	1.15	1.15	**1.20**	1.10	5.45

System	Hallucination Rate	Avg Response Time
Direct LLM	100%	~3s
RAG Baseline	25%	~45s
RAG Baseline	**5%**	~3s

RAG Baseline (5.60/6): Fixed retrieval sequence provides stable, well-grounded answers with rich evidence citation. Hallucination rate of 5% demonstrates the effectiveness of retrieval augmentation over direct generation.

Agent (5.45/6): Slightly lower than RAG baseline due to occasional tool selection errors. Achieves comparable evidence quality (1.15) and groundedness (1.15) with flexible retrieval. The model occasionally calls fewer/wrong tools but recovers 75% hallucination and post-retrieval reasoning.

Direct LLM (1.35/6): Fast response (~3s) but ungrounded. Hallucinates in 100% of responses with 100% hallucination rate. Confirms that retrieval augmentation is essential for business queries. QA.

Cross-Stage Performance Summary

Stage	Best Method	Key Metric	Key Metric
Stage 1	BERT fine-tuned	5-class	Accuracy 62.58%
Stage 2	RoBERTa base (3-class, 5 epochs)	5-class	Accuracy 68.56%
Stage 3	Qwen3.5-7B LoRA (5090)	5-class	Accuracy 69.39%
Stage 4	RAG Baseline	Open-ended QA	Score 5.60/10, Hallucination 5%

The classification accuracy ceiling reflects the inherent ambiguity in star rating prediction (adjacent labels are subjectively similar). Stage 4 shifts the evaluation from accuracy to groundedness, where RAG grounding pays off. Stage 3 results suggest that scaling the problem as open-ended business intelligence, where retrieval augmentation and structured tool use provide the largest qualitative leap.

Portfolio Highlights

- **RAG pipeline design:** Dual-mode RAG indexing, business-filtered pre-retrieval, evidence-grounded synthesis
- **LangGraph agent implementation:** ReAct loop, tool binding with `create_react_agent`, message trace serialization
- **Tool design for small LLMs:** Decomposing retrieval scope into two focused tools improves 7B model reliability under ReAct prompting
- **Structured evaluation:** rubric design across 3 dimensions, three-way system comparison, hallucination rate as a binary metric
- **Production readiness:** crash-resume eval runner, multi-layer JSON parsing, centralized config management
- **Interactive demos:** Gradio 5 app with 100 business searchable dropdown, five tool calls, and side-by-side evidence display
- **Systematic NL experimentation:** 216 run hyperparameter sweep with automated logging and result aggregation

CS4641 Text Mining Project · github.com/YuhanRongGT/Yelp Project